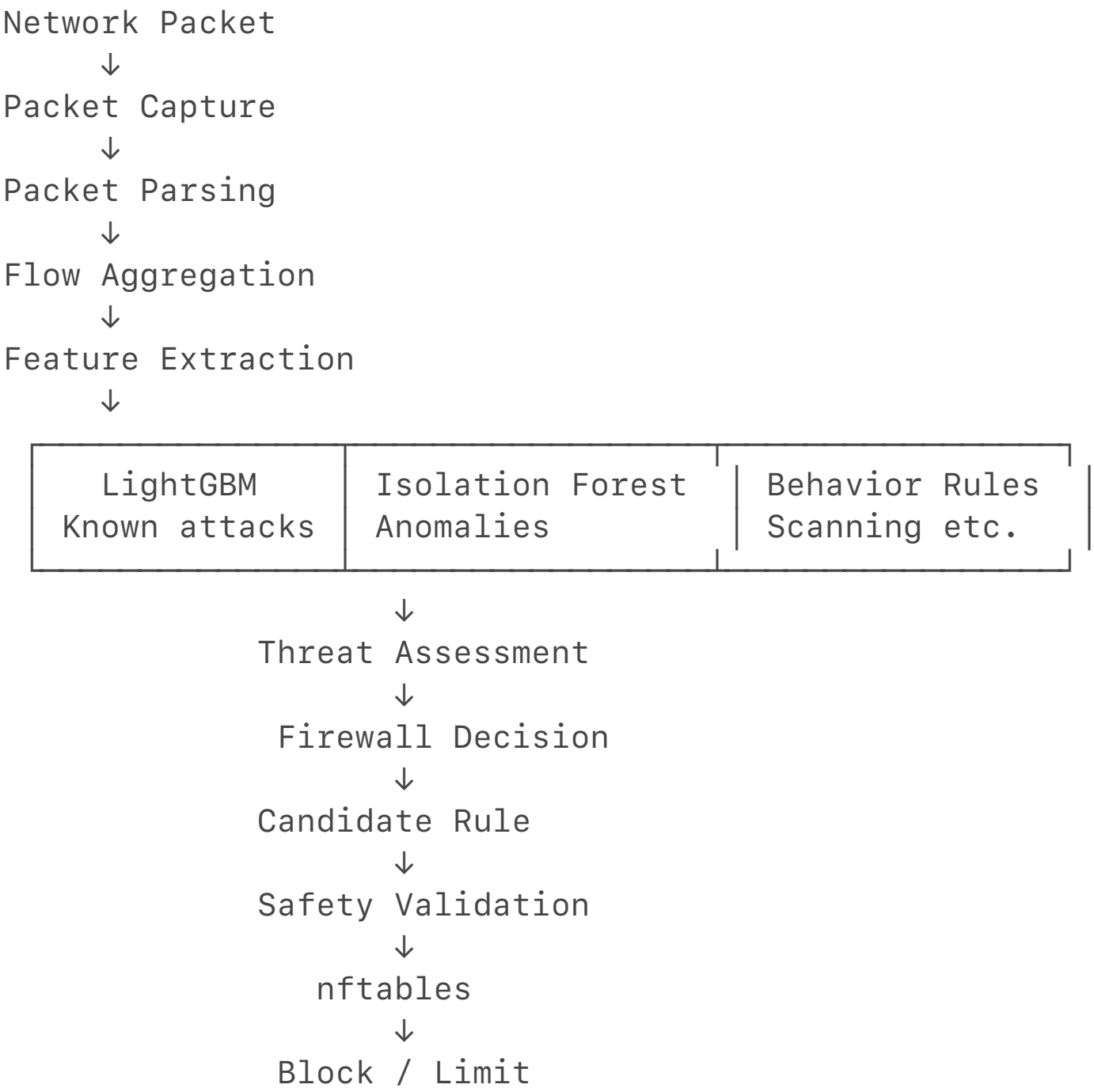
1. What is this project?

pirewall = AI-assisted adaptive network firewall designed for a Raspberry Pi 4.

The basic idea is:

Put a Raspberry Pi between your protected LAN and the outside network, inspect network traffic, detect suspicious behavior using AI/ML + rule-based analysis, decide how dangerous it is, and automatically create a narrowly scoped firewall rule when appropriate.

The core pipeline is:



The important design principle is:

AI does NOT directly control the firewall.

The AI produces **evidence**.

Then the system:

```
AI evidence
    ↓
Threat score
    ↓
Decision
    ↓
Candidate firewall rule
    ↓
10 safety checks
    ↓
Firewall
```

That separation is one of the strongest parts of the project.

2. What problem is it solving?

A traditional firewall mostly works using manually configured rules such as:

```
Allow TCP 443
Block IP 10.0.0.50
Allow LAN → Internet
```

Your project tries to make this more adaptive.

For example, imagine:

```
192.168.1.50
      |
      | 1 connection
      ↓
Internet server
```

Nothing unusual.

But suppose:

```
192.168.1.50
      |
      ├── port 21
      ├── port 22
      ├── port 23
      ├── port 25
      ├── port 53
      ├── port 80
      |
```

```
        ├── port 110
        ├── port 139
        ├── port 443
        └── ...
```

The behavior analyzer may recognize this as scanning.

If ML also considers the traffic suspicious, the combined threat score can become high.

Then:

```
Threat Level = HIGH
       ↓
Decision = RATE_LIMIT
       ↓
Candidate firewall rule
       ↓
Safety validation
       ↓
nftables
```

For extremely high confidence:

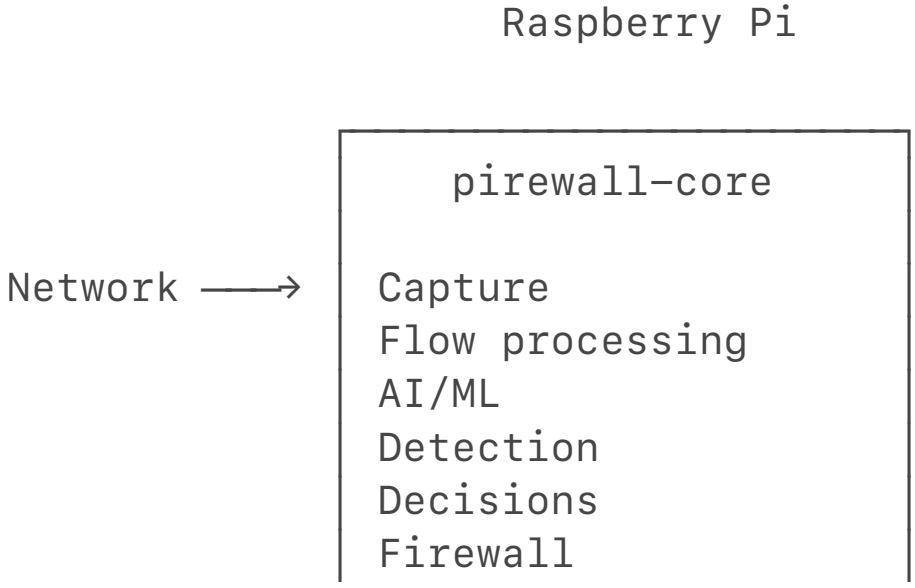
```
Threat Level = CRITICAL
       ↓
Decision = BLOCK
```

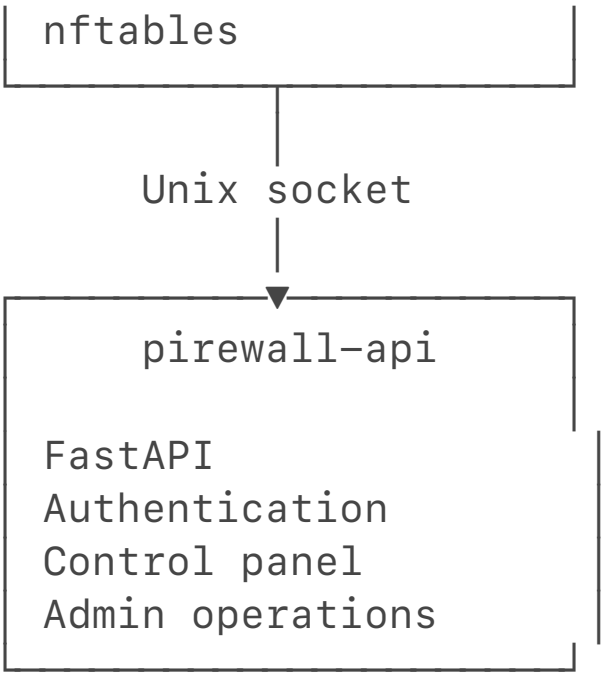
3. The project has TWO major processes

This is extremely important.

The project deliberately does **not** run everything in one process.

There are:





pirewall-core

This is the privileged process.

It has the capabilities necessary for:

- raw packet capture
- firewall management

It contains:

- capture
- flow aggregation
- feature extraction
- ML
- detection
- threat scoring
- decisions
- rule generation
- nftables
- runtime management

pirewall-api

This is the unprivileged process.

It handles:

- login
- sessions
- API endpoints
- dashboard
- admin actions

It **cannot directly access nftables**.

Instead:

```
Browser
   ↓
FastAPI
   ↓
RPC
   ↓
pirewall-core
   ↓
FirewallManager
   ↓
nftables
```

This protects the firewall if the web application is compromised.

4. Folder structure

The important structure is:

```
pirewall-main/
│
├── config/
├── deploy/
├── docs/
├── pirewall/
├── prompts/
├── scripts/
├── tests/
│
├── README.md
├── CLAUDE.md
└── pyproject.toml
```

The actual application is inside:

```
pirewall/
```

5.

`pirewall/`

— the actual application

Inside:

```
pirewall/
├── api/
├── capture/
├── config/
├── core/
├── detection/
├── engine/
├── features/
├── firewall/
├── flow/
├── integration/
├── ipc/
├── ml/
├── runtime/
└── web/
```

Each directory has a very specific responsibility.

6.

`capture/`

— capturing network packets

Files:

```
capture/
├── af_packet.py
├── fake.py
├── interfaces.py
├── parser.py
└── pipeline.py
```

`af_packet.py`

This is the real Linux packet capture implementation.

It uses:

`socket.AF_PACKET`

This allows the Raspberry Pi to capture Ethernet frames directly.

It requires Linux capabilities such as:

`CAP_NET_RAW`

It can capture packets from the selected interface.

`fake.py`

This is a fake packet capture implementation used for testing.

Instead of:

```
Real network
    ↓
Ethernet card
    ↓
AF_PACKET
```

tests can do:

```
Fake packets
    ↓
FakePacketCapture
```

This is important because you don't need a Raspberry Pi every time you run unit tests.

7.

`parser.py`

— turning raw packets into usable information

A raw packet looks roughly like:

```
Ethernet header
      ↓
IP header
      ↓
TCP/UDP/ICMP header
      ↓
Payload
```

The parser extracts information such as:

```
Source IP
Destination IP
Source port
Destination port
Protocol
Packet length
TCP flags
Timestamp
```

For example:

```
192.168.1.20:51234
        ↓
10.0.0.5:443
        ↓
TCP
```

becomes something like:

```
source_ip = 192.168.1.20
destination_ip = 10.0.0.5
source_port = 51234
destination_port = 443
protocol = TCP
```

It supports:

- IPv4
- IPv6 parsing
- TCP
- UDP
- ICMP
- ICMPv6

However, the actual flow system is intentionally **IPv4-only**.

That's an important limitation.

8.

`flow/`

— turning packets into flows

A firewall doesn't necessarily want to run the AI model independently on every packet.

Instead, packets are grouped into **flows**.

For example:

```
Packet 1
192.168.1.10:50001 → 8.8.8.8:53

Packet 2
192.168.1.10:50001 → 8.8.8.8:53

Packet 3
192.168.1.10:50001 → 8.8.8.8:53
```

These belong to one flow:

```
Flow
source       = 192.168.1.10
destination  = 8.8.8.8
source port  = 50001
dest port    = 53
protocol     = UDP
packets      = 3
bytes        = ...
duration     = ...
```

`flow/aggregator.py`

The FlowAggregator receives packets and updates the corresponding flow.

It also handles flow completion.

For TCP, things like:

```
FIN
RST
```

can cause a flow to finish.

There are also timeouts:

```
active timeout
inactive timeout
```

9. Why flow aggregation matters

The AI needs more useful information than:

```
"This packet is 74 bytes."
```

A flow lets it see patterns like:

```
100 packets
10,000 bytes
2 seconds
95 packets forward
5 packets backward
20 SYN packets
```

That is much more useful for attack detection.

10. Resource limits

The project is designed for a Raspberry Pi, so memory cannot be allowed to grow forever.

The configuration has:

```
max_flows = 100000
```

The flow table is bounded.

If it fills up, old flows can be evicted.

This protects against a malicious client generating huge numbers of flows.

11.

`features/`

— converting flows into ML features

This is where the flow becomes a numerical feature vector.

The project defines **29 canonical features**.

For example:

```
duration_seconds
packet_count
byte_count
forward_packet_count
backward_packet_count
forward_byte_count
backward_byte_count
packets_per_second
bytes_per_second
mean_packet_size
std_packet_size
min_packet_size
max_packet_size
...
```

It also includes TCP flags:

```
syn_count
ack_count
fin_count
rst_count
psh_count
urg_count
```

And protocol indicators:

```
protocol_is_tcp
protocol_is_udp
protocol_is_icmp
protocol_is_other
```

12. Why the feature schema is important

There is a central file:

```
pirewall/features/schema.py
```

It defines:

```
SCHEMA_VERSION = 1.0.0
```

and the exact feature ordering.

This prevents a major ML problem.

Imagine the model was trained with:

```
1 = packet count
2 = byte count
3 = duration
```

but runtime accidentally sends:

```
1 = duration
2 = packet count
3 = byte count
```

The model would still produce an answer, but the answer would be garbage.

So the project treats the feature schema as a contract.

13. The AI has TWO models

This is probably the most important part for your project presentation.

There are:

1. LightGBM

Used for:

Known-attack classification

2. Isolation Forest

Used for:

Anomaly detection

They do different jobs.

14. LightGBM

LightGBM is a supervised machine-learning model.

It learns from labeled examples.

Conceptually:

```
Training data

Normal traffic       → BENIGN
Port scan            → ATTACK
DoS traffic          → ATTACK
Other attack         → ATTACK
```

The model learns patterns from the 29 features.

At runtime:

```
Flow
 ↓
29 features
 ↓
LightGBM
 ↓
Predicted class
 ↓
Confidence
```

Example:

```
predicted_class = port_scan
confidence = 0.94
```

That becomes:

```
KnownEvidence
```

15. Isolation Forest

Isolation Forest works differently.

It is an anomaly detector.

Instead of asking:

“Which known attack is this?”

it asks:

“Does this traffic look unusual compared with normal traffic?”

So:

```
Normal-looking traffic
        ↓
not anomaly

Unusual traffic
        ↓
anomaly
```

The system stores:

```
anomaly_score
threshold
is_anomaly
```

Important:

An anomaly is not automatically considered malicious.

It is only evidence.

16. Why use two models?

Because each model catches different things.

Imagine:

```
LightGBM:
"This looks like a known attack."
```

while:

```
Isolation Forest:
"This is highly unusual."
```

Both agree.

That's stronger evidence than relying on only one.

The project therefore combines:

```
Known attack evidence
+
Anomaly evidence
+
Behavior evidence
```

17.

`detection/behavior.py` — non-ML detection

This is another major component.

The project doesn't rely completely on AI.

It also has deterministic behavioral analysis.

For example:

Repeated connections

```
Same source
→ same destination
→ many repeated connections
```

Port scanning

```
One source
→ many ports
```

Destination diversity

```
One source
→ many destinations
```

High-frequency traffic

```
Many connections per second
```

Burst behavior

```
10 connections
within 5 seconds
```

Failed connections

```
Many flows
with no response
```

Persistence

Long-lasting suspicious behavior.

18. Why use deterministic behavior rules?

Because ML isn't perfect.

Suppose the model says:

```
Not sure
```

but the behavior analyzer sees:

```
Source scanned 25 ports in a few seconds.
```

That's strong evidence.

So the system uses:

```
ML + deterministic behavioral analysis
```
rather than ML alone.

19. Bounded behavior state

The behavior analyzer tracks sources.

But it can’t store information about every IP forever.

The configuration limits:

```
max_tracked_sources
max_tracked_destinations_per_source
max_tracked_ports_per_source
recent_connections_window
```
When necessary, old source state is evicted.

Again, this protects against memory exhaustion.

20.

`engine/`
— converting evidence into a threat score

This is where everything becomes a number.

The project uses a **0–100 threat score**.

Default weights are:

```
Known attack = 50
Anomaly      = 25
Behavior     = 25
```
So maximum:

```
50 + 25 + 25 = 100
```

21. Example threat-score calculation

Suppose LightGBM says:

```
Attack confidence = 0.9
```
Known-attack contribution:

```
50 × 0.9 = 45
```
Isolation Forest says:

```
Anomaly = true
```
Contribution:

```
25
```
Behavior analyzer detects several patterns and contributes, say:

```
12.5
```
Total:

```
45 + 25 + 12.5
= 82.5
```
Therefore:

```
Threat Score = 82.5
```

22. Threat levels

The default configuration defines:

```
0–24.99     LOW
25–49.99    MEDIUM
50–74.99    HIGH
75–100      CRITICAL
```
More precisely, the code checks the configured thresholds:

```
critical = 90
high     = 75
medium   = 50
```
So:

```
30 → MEDIUM
60 → HIGH
82 → HIGH
95 → CRITICAL
```

23.

`engine/decision.py`

Now the threat level becomes a firewall action.

The project uses:

Threat level	Action
LOW	ALLOW

MEDIUM	MONITOR
HIGH	RATE_LIMIT
CRITICAL	BLOCK

So:

```
LOW
 ↓
ALLOW
MEDIUM
 ↓
MONITOR
HIGH
 ↓
RATE LIMIT
CRITICAL
 ↓
BLOCK
```

This is deliberately simple and explainable.

24. Why LOW doesn't create a firewall rule

Because there's no reason to add a rule for normal traffic.

So:

```
LOW → ALLOW → no candidate rule
```

This keeps the firewall from filling up with unnecessary rules.

25.

`firewall/generator.py`

Suppose we have:

```
Source:
192.168.1.50

Destination:
10.10.10.20
```

```
Destination port:
22
```

The generator creates a very narrow rule:

```
source      = 192.168.1.50/32
destination = 10.10.10.20/32
protocol    = TCP
destination port = 22
```

/32 means:

exactly one IPv4 address.

This is important.

The system doesn't say:

```
Block 192.168.0.0/16
```

based on one suspicious flow.

It tries to block only what the evidence actually supports.

26. Why source port isn't included

The generator deliberately doesn't normally match the attacker's source port.

Why?

Because client source ports are usually ephemeral.

For example:

```
192.168.1.50:50001 → server:22
192.168.1.50:50002 → server:22
192.168.1.50:50003 → server:22
```

If the rule blocked only:

```
source port 50001
```

the attacker could simply use:

```
50002
```

So the rule focuses on:

```
source IP
destination IP
protocol
destination port
```

27. The 10-stage firewall validation

This is one of the best security features in the code.

Before a candidate rule reaches nftables, it passes:

1. Schema
2. Network
3. Allowlist
4. Safety
5. Conflict
6. Duplicate
7. Rate cap
8. Priority
9. Expiration
10. Authorization

Only after passing all ten can it be deployed.

28. Safety validation

The safety stage prevents the adaptive firewall from accidentally destroying the network.

For example, it prevents a rule from blocking:

The Admin PC

Otherwise you could lock yourself out.

The Raspberry Pi itself

Otherwise you could block the firewall's own management address.

The upstream gateway

Otherwise the LAN could lose Internet access.

The entire protected network

A single suspicious flow shouldn't produce:

```
BLOCK 192.168.1.0/24
```

Entire Internet

It also rejects:

```
0.0.0.0/0
```

for restrictive adaptive rules.

Overly broad prefixes

The default minimum prefix length is:

```
/24
```

meaning adaptive rules can't be broader than that.

But the generator itself uses /32, which is even narrower.

29. Static allowlist

The project has an allowlist.

Example:

```
Admin PC
Home server
Important device
```

If something is on the allowlist:

```
Adaptive BLOCK
       ↓
REJECT
```

The allowlist has priority over adaptive restrictive rules.

This is very important because otherwise the AI could theoretically decide:

```
Admin PC looks suspicious
→ BLOCK
```

The allowlist prevents that.

30. Rule rate limiting

Imagine an attack creates:

```
10,000 suspicious flows
```

The firewall shouldn't create:

```
10,000 firewall rules
```

The project therefore has:

```
max_adaptive_rules_per_window = 20
rate_window_seconds = 60
```

So rule creation is capped.

Important distinction:

Detection doesn't stop.

Only:

```
rule creation
```

is limited.

The system can still detect and record suspicious activity even when the rule budget is exhausted.

31. Rule expiration

Adaptive rules have a TTL.

Default:

```
3600 seconds
```

That's one hour.

So an adaptive rule can go:

```
ACTIVE
   ↓
one hour passes
   ↓
EXPIRED
```

This prevents temporary attacks from creating permanent firewall rules.

32. Shadow mode

The default enforcement mode is:

```
SHADOW
```

This is extremely useful for deployment.

In shadow mode:

```
Detection
   ↓
Decision
   ↓
Candidate rule
   ↓
Validation
   ↓
NOT actually deployed
```

Instead, the system records:

"I would have blocked this."

This allows administrators to observe the system before trusting it with real traffic.

33. Assisted mode

In assisted mode, high-risk actions can require human approval.

Conceptually:

```
Threat detected
      ↓
Candidate rule
      ↓
Validation
      ↓
PENDING_APPROVAL
      ↓
Admin reviews
      ↓
APPROVE
      ↓
Deploy
```

or:

```
REJECT
```

34. Active enforcement

Eventually:

```
enforcement_mode = ACTIVE
```

Then approved candidates can actually be deployed.

35. Rule lifecycle

The project tracks rule states.

Something can go:

```
CANDIDATE
    ↓
VALIDATING
    ↓
PENDING_APPROVAL
    ↓
```

```
APPROVED
    ↓
DEPLOYED
    ↓
ACTIVE
```

Or:

```
CANDIDATE
    ↓
REJECTED
```

Or:

```
ACTIVE
 ↓
EXPIRED
```

Or:

```
ACTIVE
 ↓
DISABLED
```

Or:

```
ACTIVE
 ↓
REMOVED
```

Every transition is recorded for auditing.

36.

nftables

The real firewall backend is:

`pirewall/firewall/backend/nftables.py`

It uses Linux **nftables**.

The architecture creates:

`inet pirewall`

with:

`adaptive chain`

The project communicates with nftables using its JSON interface.

It deliberately avoids doing something dangerous like:

```
os.system("nft ... " + user_input)
```

Instead it builds structured JSON and runs:

```
nft -j -f -
```
with shell=False.

That is much safer.

37. What happens for BLOCK?

A BLOCK rule becomes essentially:

```
match source
match destination
match protocol/port
      ↓
DROP
```
So traffic matching that rule is dropped.

38. RATE_LIMIT

Rate limiting is slightly more complicated.

The project creates two rules conceptually:

```
If traffic is within rate:
    ACCEPT

Otherwise:
    DROP
```
This is important because simply applying an nftables limit does not automatically mean excess traffic is dropped.

39. MONITOR

Monitor rules don’t block traffic.

They can:

```
LOG
+
COUNTER
```
So the system can observe suspicious traffic without immediately disrupting it.

40.

`FirewallManager`

This is the **single authorized gateway** to the firewall backend.

Other parts of the project cannot simply say:

```
nft block this IP
```

Instead:

```
Detection
 ↓
Decision
 ↓
CandidateRule
 ↓
FirewallManager
 ↓
Validation
 ↓
Backend
```

This is an excellent security boundary.

41. Kill switch

There is an emergency kill switch.

Conceptually:

```
Admin
 ↓
Kill switch
 ↓
SHADOW mode
 ↓
Remove active adaptive rules
 ↓
Return to static base rules
```

This does **not** remove the static base firewall.

It removes adaptive rules.

So if the AI starts behaving badly, the administrator has a way to rapidly revert the adaptive portion.

42.

`ipc/`

— communication between the two processes

The two processes communicate through:

`AF_UNIX`

Unix domain sockets.

The API doesn't directly import:

`firewall backend`

Instead it sends something like:

```
operation = get_status
```

or:

```
operation = disable_rule
params = {
    "rule_id": "..."
}
```

The core receives it.

43. Why RPC instead of direct imports?

Imagine the API process were compromised.

If it directly had:

```
from pirewall.firewall.backend import ...
```

the attacker could potentially gain a direct path to firewall control.

Instead:

```
API
 ↓
small predefined RPC protocol
 ↓
CORE
```

The RPC protocol has a closed list of operations.

For example:

```
GET_STATUS
LIST_FLOWS
LIST_THREATS
```

```
LIST_RULES
DISABLE_RULE
REMOVE_RULE
APPROVE_RULE
REJECT_RULE
KILL_SWITCH
...
```

There is no:

```
EXECUTE_ARBITRARY_COMMAND
```

44.

`api/`

— FastAPI

The API is built with FastAPI.

It exposes endpoints such as:

```
/api/v1/health
/api/v1/status
/api/v1/flows
/api/v1/detections
/api/v1/threats
/api/v1/decisions
/api/v1/rules
/api/v1/events
/api/v1/models
```

Administrative actions include:

```
/rules/{id}/disable
/rules/{id}/remove
/rules/{id}/approve
/rules/{id}/reject
```

and:

```
/firewall/kill-switch
```

45. Authentication

The project doesn’t store the admin password directly.

It uses:

`hashlib.scrypt`

to derive a password hash.

Conceptually:

```
Password
   ↓
Random salt
   ↓
scrypt
   ↓
Password hash
```

When logging in:

```
Entered password
       ↓
scrypt
       ↓
Compare with stored hash
```

The comparison uses:

`hmac.compare_digest`

to avoid timing-related comparison issues.

46. Sessions

After successful login, the API creates a random session token.

It isn't using JWT.

Instead:

`random opaque token`

is stored in an in-memory session store.

Sessions expire after the configured time.

47. Admin PC restriction

The project has another important security control:

`admin_pc_ip`

Only the configured administrator PC is allowed to access administrative functionality.

So there are effectively two checks:

```
Is this the Admin PC?
        AND
Is the user authenticated?
```

Both must pass.

48. Control panel

The project has a web control panel.

It can display things like:

```
System status
Threats
Firewall rules
Events
ML status
```

The API also has an event stream using **Server-Sent Events (SSE)**.

So the dashboard can receive events such as:

```
THREAT_DETECTED
RULE_REJECTED
MODEL_ERROR
FLOW_ERROR
```

without repeatedly refreshing the page.

49. Security events

The project has a common:

```
SecurityEvent
```

model.

This is important because different components can generate events.

For example:

```
Detection
    ↓
SecurityEvent

Firewall
    ↓
SecurityEvent
```

```
Authentication
    ↓
SecurityEvent

System
    ↓
SecurityEvent
```

These can then be:

```
stored locally
      +
sent to Wazuh
      +
shown in control panel
```

50. Wazuh integration

The project integrates with Wazuh.

The purpose is:

Send security events from pirewall to a centralized SIEM.

For example:

```
pirewall detects port scan
        ↓
SecurityEvent
        ↓
JSON
        ↓
Wazuh
        ↓
SIEM dashboard/correlation
```

The project sends to the **remote syslog collector**, configured around port:

```
514
```

not the Wazuh agent port 1514.

The actual delivery to a real Wazuh installation is still environment-dependent.

51. Netdata integration

Netdata is used for operational monitoring.

The project can export metrics such as:

```
CPU
memory
packet rate
packet drops
active flows
flow creation rate
inference count
inference latency
detection count
block count
rule count
rule rejection count
API health
capture health
firewall health
```

These are sent through StatsD/UDP.

So:

```
pirewall
   ↓
metrics
   ↓
Netdata
   ↓
Admin monitoring
```

52.

runtime/

This is essentially the glue that makes the whole system actually run.

Important files:

```
runtime/
├── core.py
├── forwarder.py
├── metrics.py
├── pipeline.py
```

```
└── watchdog.py
```

53.

`runtime/pipeline.py`

This is probably the single most useful file to understand for your viva.

It connects everything.

For every completed flow:

```
Flow
 ↓
extract_features()
 ↓
DetectionCoordinator
 ↓
assess_threat()
 ↓
decide()
 ↓
generate_candidate_rule()
 ↓
FirewallManager
```

So this is where the architecture becomes an actual working pipeline.

54. Full example

Let's follow an imaginary attack.

Suppose:

```
Attacker:
192.168.1.50

Target:
192.168.1.100
```

The attacker performs many connections.

Step 1 — packet capture

`AF_PACKET`
captures the Ethernet frames.

Step 2 — parser

The parser extracts:

```
source = 192.168.1.50
destination = 192.168.1.100
protocol = TCP
destination port = 22
flags = SYN
```

Step 3 — flow aggregation

Multiple packets become:

```
Flow #123
```

with statistics.

Step 4 — feature extraction

The system calculates 29 features.

For example:

```
packet_count = 30
syn_count = 25
packets_per_second = 10
...
```

Step 5 — LightGBM

It might produce:

```
predicted_class = attack
confidence = 0.90
```

Step 6 — Isolation Forest

It might produce:

```
anomaly_score = negative
is_anomaly = true
```

Step 7 — behavior analyzer

It might detect:

```
REPEATED_CONNECTIONS
HIGH_FREQUENCY
```

Step 8 — threat scoring

For example:

```
Known attack = 45
Anomaly      = 25
Behavior     = some contribution

Total        = 80+
```

So:

```
Threat Level = HIGH
```

or possibly CRITICAL depending on the exact evidence.

Step 9 — decision

If HIGH:

```
HIGH
 ↓
RATE_LIMIT
```

Step 10 — candidate rule

Something like:

```
source = 192.168.1.50/32
destination = 192.168.1.100/32
protocol = TCP
destination port = 22
action = RATE_LIMIT
```

Step 11 — validation

The candidate passes:

```
Schema ✓
Network ✓
Allowlist ✓
Safety ✓
Conflict ✓
Duplicate ✓
Rate cap ✓
```

```
Priority ✓
Expiration ✓
Authorization ✓
```

Step 12 — enforcement

If active mode is enabled:

```
FirewallManager
       ↓
NftablesBackend
       ↓
nftables
```

The firewall rule is installed.

55. What happens if something goes wrong?

This is another strong design feature.

Suppose ML inference crashes.

The entire firewall does **not** crash.

Instead:

```
LightGBM error
     ↓
MODEL_ERROR
     ↓
LightGBM evidence = unavailable
     ↓
continue with Isolation Forest + behavior
```

Similarly, if an individual flow causes an error:

```
Flow error
 ↓
SecurityEvent
 ↓
drop that flow
 ↓
continue processing other traffic
```

56. Capture and detection are separate threads

This is important for Raspberry Pi performance.

The capture thread needs to receive packets quickly.

But ML inference can take significantly longer.

So:

```
Capture thread
      ↓
bounded queue
      ↓
Detection thread
```

If detection becomes slow, the capture thread shouldn't sit there waiting.

The queue is bounded so memory cannot grow forever.

57. Watchdog

The runtime also has systemd/watchdog support.

The idea is:

```
pirewall-core
      ↓
watchdog heartbeat
      ↓
systemd
```

If the core crashes, systemd can restart it.

The project also considers crash loops.

58. Fail-open behavior

The default is:

```
fail_open
```

This is an important design choice for a home network.

Suppose:

```
pirewall-core crashes
```

The adaptive rules could remain in the Linux kernel.

That could mean:

```
pirewall stopped
but traffic still blocked
```

The project therefore removes adaptive rules during clean shutdown in fail-open mode and

returns to the base ruleset.

The philosophy is:

A crashed AI firewall shouldn’t permanently cut off the household network.

59.

`config/`

The main configuration is:

`config/default_config.toml`

It controls:

`network`
`capture`
`flow`
`features`
`detection`
`ML`
`threat scoring`
`firewall`
`API`
`authentication`
`admin PC`
`logging`
`Wazuh`
`Netdata`
`security`
`failure behavior`

The default file intentionally contains placeholders such as:

`CHANGE_ME`

because real network details and secrets should not be committed.

A real deployment should use:

`config/local_config.toml`

which is gitignored.

60. Important configuration values

Network

```
wan_interface
lan_interface
protected_network
upstream_gateway
pirewall_lan_ip
```

These define how the Raspberry Pi is positioned as a gateway.

Capture

```
interface
snap_len
promiscuous
buffer_size_bytes
```

Flow

```
active_timeout_seconds
inactive_timeout_seconds
max_flows
```

Detection

```
known_attack_confidence_threshold
anomaly_score_threshold
behavior_window_seconds
```

plus all the behavior limits.

Threat

```
known_attack_weight = 50
anomaly_weight = 25
behavior_weight = 25
```

Firewall

```
enforcement_mode = shadow
max_adaptive_rules_per_window = 20
max_active_rules = 500
default_rule_ttl_seconds = 3600
```

61.

deploy/

This folder contains things needed for actual Raspberry Pi deployment.

```
deploy/
├── certificates/
├── firewall/
├── network/
└── systemd/
```

62.

deploy/firewall/

Contains:

`base.nft.template`

This establishes the base firewall rules.

The important concept is that there are two layers:

```
Static base rules
+
Adaptive rules
```

The adaptive rules get evaluated first so that a narrow adaptive block can override broader static forwarding behavior.

63.

deploy/network/

Contains network configuration templates.

For example:

`60-pirewall-forwarding.conf.template`

controls kernel forwarding behavior.

There are also templates for:

```
DHCP/network interface configuration
NAT masquerading
```

64. NAT

The Raspberry Pi is intended to operate as a gateway.

Conceptually:

```
LAN devices
     ↓
Raspberry Pi
     ↓
WAN
     ↓
Internet
```

NAT allows private LAN addresses to communicate outward through the upstream interface.

65.

deploy/systemd/

There are two service files:

```
pirewall-core.service
pirewall-api.service
```

The core gets:

```
CAP_NET_RAW
CAP_NET_ADMIN
```

while the API gets:

```
no firewall/raw-socket capabilities
```

The systemd units also include hardening such as:

```
NoNewPrivileges
ProtectSystem
PrivateTmp
resource limits
restricted paths
restricted address families
```

66.

docs/

This project has a LOT of documentation.

Important ones:

```
MASTER_SPEC.md
ARCHITECTURE.md
ML_PIPELINE.md
FEATURE_SCHEMA.md
```

FIREWALL.md
SECURITY.md
DEPLOYMENT.md
SETUP.md
TESTING.md
PROGRESS.md

MASTER_SPEC.md

Original project specification.

ARCHITECTURE.md

Explains module boundaries and data flow.

ML_PIPELINE.md

Explains:

```
datasets
→ preprocessing
→ training
→ artifacts
→ inference
```

FIREWALL.md

Explains:

```
decision
→ candidate
→ validation
→ lifecycle
→ nftables
```

SECURITY.md

Explains the threat model and hardening.

DEPLOYMENT.md

Explains how to put it on a real Pi.

TESTING.md

Explains all test categories.

PROGRESS.md

Probably the most important file for determining what is **actually complete**.

67.

scripts/

These are helper programs.

There are deployment tools:

`scripts/deployment/`

including:

`configure.py`
`discovery.py`
`render_templates.py`
`make_certs.sh`

They help discover the network and generate deployment configuration.

There are also training scripts:

`scripts/train/`

for:

`train_lightgbm.py`
`train_isolation_forest.py`

And:

`scripts/diagnostics/`

for performance testing.

68.

`tests/`

This project has a very large test suite.

There are:

`tests/unit/`
`tests/integration/`
`tests/security/`
`tests/ml/`
`tests/system/`

This is not just testing whether individual functions work.

It tests things like:

`packet parsing`
`flow aggregation`
`ML inference`
`threat scoring`
`firewall validation`
`authentication`
`API routes`
`RPC`

```
security isolation
resource exhaustion
systemd hardening
```

69. Security tests

Some particularly important security tests check:

```
test_api_process_isolation.py
test_backend_isolation.py
test_injection.py
test_resource_exhaustion.py
test_safety_validation.py
test_firewall_failure_handling.py
test_systemd_hardening.py
```

These are specifically trying to ensure that the architecture's security promises are actually enforced by tests.

70. What is actually complete?

According to the project's own progress tracking, the software implementation is **quite far along**.

The project reports completed/tested phases for:

```
Packet capture & parsing
Flow/features
Dataset/ML pipeline
ML inference
Behavior analysis
Threat assessment
Firewall decisions
Rule validation
nftables implementation
API
Authentication
Control panel
RPC
Security events
Deployment tooling
```

```
Testing
```

However, there's a very important distinction.

71. "Implemented" does NOT mean "verified on Raspberry Pi"

Several components are implemented but still require real hardware/environment verification.

Most importantly:

Real packet capture

```
AF_PACKET
```

needs a real Linux machine/interface and appropriate capabilities.

Real nftables

Needs:

```
Linux
nft binary
CAP_NET_ADMIN/root
```

Real systemd hardening

Needs the actual Pi/systemd environment.

Real TLS certificates

Need to be deployed and tested.

Real Wazuh

Needs an actual Wazuh server.

Real Netdata

Needs an actual Netdata installation.

Real ML accuracy

This is especially important.

The code can run ML inference, but **real attack-detection accuracy has not been validated against a real attack dataset/environment in this repository**.

72. The ML models are not the same thing as proving the AI works

This is something you should be very clear about in your presentation.

The project contains the ML infrastructure:

```
Dataset adapters
↓
Preprocessing
```

```
↓
Training
↓
LightGBM
↓
Isolation Forest
↓
Model metadata
↓
Runtime inference
```

But the project’s own progress documentation says the real detection performance remains environment-dependent.

In other words:

The ML pipeline is implemented, but you shouldn’t claim that the AI has been proven highly accurate against real attacks unless you actually run it on appropriate real attack data.

73. Another important limitation: IPv6

The parser can understand IPv6 packets.

But the flow layer intentionally doesn’t create IPv6 flows.

So effectively:

```
IPv4
 ↓
Flow
 ↓
Features
 ↓
Detection
```

but:

```
IPv6
 ↓
parsed/counted
 ↓
not converted into a Flow
 ↓
doesn't reach ML
```

The project explicitly chooses IPv4-only operation for this version.

74. Another limitation: VLAN

The parser doesn't support:

```
802.1Q VLAN tags
```

So a VLAN-tagged Ethernet frame can be treated as unsupported.

That's documented rather than hidden.

75. The control-panel gaps

The progress documentation also identifies some limitations.

For example, the API has:

```
GET /api/v1/detections
```

but the HTML control panel doesn't currently render a dedicated detections section.

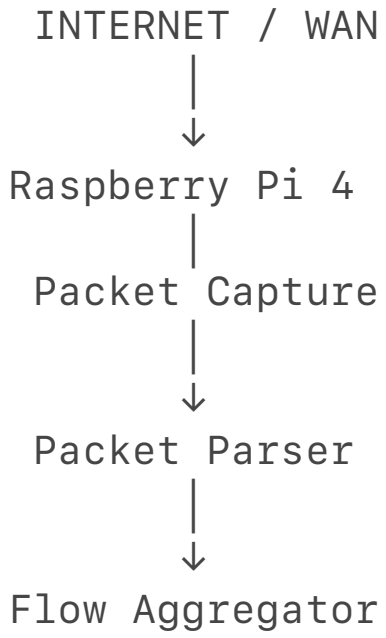
There was also a documented gap around network statistics in the control panel, although the runtime metrics plumbing itself exists.

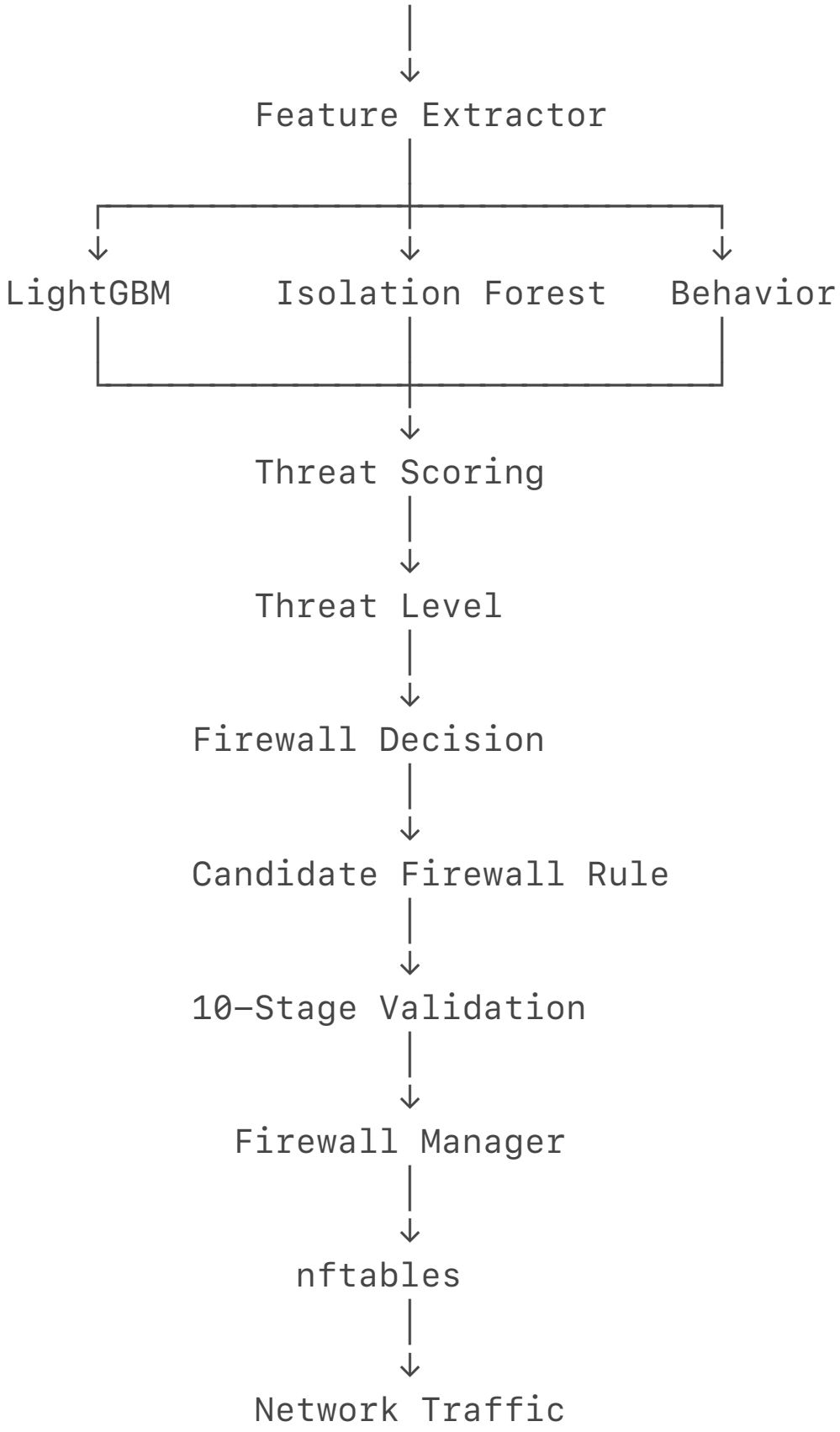
So:

```
API functionality ≠ every feature necessarily displayed in UI
```

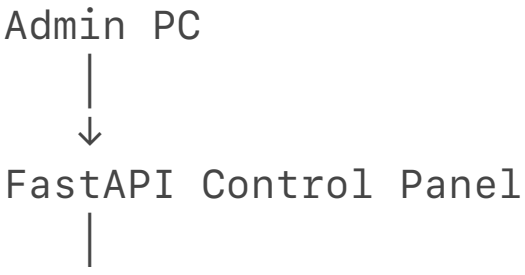
76. The most important architecture diagram to remember

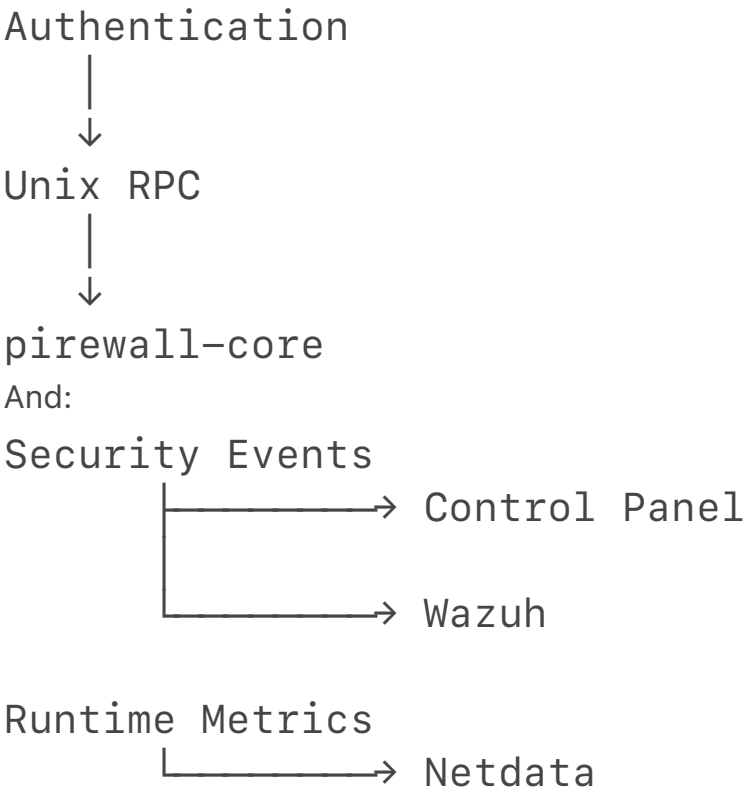
For your viva, memorize this:





And separately:





77. What makes this different from a normal firewall?

The key difference is **adaptive detection**.

A normal firewall says:

```
Rule 1
Rule 2
Rule 3
...
```

Your system does:

```
Observe traffic
      ↓
Understand behavior
      ↓
Extract features
      ↓
Use ML
      ↓
Corroborate with behavior
      ↓
Calculate threat score
```

```
      ↓
Decide action
      ↓
Safely generate rule
      ↓
Deploy temporarily
```

So the firewall can respond to traffic patterns rather than relying exclusively on manually written rules.

78. Why Raspberry Pi?

The project is designed specifically as a **low-cost network security appliance**.

Instead of requiring:

```
expensive dedicated firewall hardware
```

the idea is:

```
Raspberry Pi 4
+
Linux
+
nftables
+
ML
+
IDS-style detection
+
SIEM integration
```

This makes it suitable conceptually for:

- home networks
- small organizations
- IoT environments
- small offices

79. One thing to be careful saying: it isn't Suricata

Based on the ZIP you gave me, **this particular repository does not use Suricata as its detection engine**.

Your earlier project description mentioned:

```
Suricata IDS/IPS
Wazuh
Netdata
Raspberry Pi
```

but this codebase's actual detection architecture is:

```
LightGBM
+
Isolation Forest
+
Deterministic Behavior Analyzer
```

and enforcement is:

```
nftables
```

So if you're presenting **this repository**, don't describe Suricata as though it is implemented inside this codebase unless you have another version/project that integrates it.

80. In one sentence, what does each major module do?

Module	Purpose
capture	Captures and parses network packets
flow	Groups packets into network flows
features	Converts flows into 29 ML features
ml	Trains/loads/runs ML models
detection	Produces known-attack, anomaly and behavioral evidence
engine	Converts evidence into threat scores and decisions
firewall	Generates, validates and deploys

	firewall rules
ipc	Secure communication between core and API
api	REST API and administration
web	Control-panel interface
runtime	Runs the complete pipeline
integration	Wazuh and Netdata communication
config	Central configuration
deploy	Raspberry Pi/network/systemd deployment
scripts	Setup, training and diagnostics
tests	Unit, integration, ML, system and security tests
docs	Full project specification/ documentation

81. The entire project in very simple language

If you need to explain it to a professor who doesn't want to hear every Python file:

pirewall is an AI-assisted adaptive firewall running on a Raspberry Pi. It captures network packets, groups them into flows, extracts traffic features, and analyzes those features using LightGBM, Isolation Forest, and deterministic behavioral rules. The resulting evidence is combined into a 0–100 threat score. Depending on the threat level, the system allows, monitors, rate-limits, or blocks the traffic. Before any adaptive rule is installed, it passes through multiple safety and authorization checks. Validated rules are deployed through nftables. The system separates the privileged firewall process from the unprivileged FastAPI control panel using a Unix-domain RPC interface. Security events can be sent to Wazuh and

operational metrics to Netdata. Shadow mode, allowlisting, rule expiration, rate limiting, resource limits, and a kill switch are included to make the adaptive firewall safer to operate.

82. The most important thing to understand for your project

Don't think of it as:

```
AI → firewall
```

Think of it as:

```
AI
 ↓
EVIDENCE
 ↓
THREAT SCORE
 ↓
DECISION
 ↓
CANDIDATE RULE
 ↓
SAFETY VALIDATION
 ↓
FIREWALL
```

That separation is essentially the **core design philosophy of the entire repository**.

And the actual end-to-end code that ties this together is primarily:

```
pirewall/runtime/pipeline.py
```

while the main privileged process is:

```
pirewall/main.py
```

and the firewall's final authority is:

```
pirewall/firewall/manager.py
```

with the actual Linux enforcement happening in:

```
pirewall/firewall/backend/nftables.py
```